

# Empirical Analysis of Supervised Learning Algorithms Across Diverse Data Domains

Anthony Mitine

COGS 118A Final Project Report

December 11, 2025

## Abstract

The selection of an appropriate classification algorithm is a challenge in machine learning. Classification algorithms are often dictated by specific dataset properties such as feature dimensionality, sample size, and noise levels. In this study, I evaluated the performance of five supervised learning classifiers, Random Forest, XGBoost, Support Vector Machines (SVM), Logistic Regression, and K-Nearest Neighbors (KNN), across four datasets of varying dimensionality and domain (Adult, Letter Recognition, Spambase, and Mushroom). Following the experiment by Caruana and Niculescu-Mizil, I analyzed the impact of training set size on generalization error. My results demonstrated that ensemble methods (XGBoost and Random Forest) consistently outperform single estimators on complex tabular data, while geometric methods (SVM and KNN) excel in spatial domains like character recognition. Furthermore, I confirmed that increasing training data volume yields diminishing returns, with the most significant gains observed in lower-data regimes.

## 1. Introduction

The “No Free Lunch” theorem in supervised machine learning states that no single algorithm is universally superior across all possible problem domains. However, empirical studies, like the one done by Caruana and Niculescu-Mizil [1], have demonstrated that certain families of algorithms, specifically ensemble methods like Bagging and Boosting, tend to provide performance on tabular datasets.

This project aims to reproduce and extend these findings by evaluating the trade-offs between algorithm complexity, training data availability, and generalization performance. I wanted to confirm that ensemble methods, such as Random Forest, & XGBoost, will consistently outperform traditional single-estimator baselines, like Logistic Regression, & KNN on

high-dimensional tabular data. I also looked into the reduction of training data from 80% to 20% and how that will disproportionately affect high-variance models compared to high-bias models.

To ensure efficiency and speed of the experiment, I utilized Google Gemini to optimize the experimental pipeline and main loop, allowing for quicker cross-validation across all trials.

## 2. Methodology

I selected five classifiers that represent distinct probabilistic, geometric, instance-based, and ensemble-based learning to conduct a comprehensive evaluation.

### 2.1 Learning Algorithms

- **Logistic Regression:** Serving as our linear baseline, Logistic Regression models the probability of a binary outcome using the sigmoid function. It assumes a linear decision boundary between classes. I optimized the regularization parameter  $C$  and tested both `lbfgs` and `liblinear` solvers.
- **Support Vector Machine (SVM):** SVMs aim to find the hyperplane that maximizes the margin between classes. I utilized the Radial Basis Function kernel, which projects data into a higher-dimensional space to handle non-linearity. This method is theoretically strong for geometric data. I tuned the penalty parameter  $C$ .
- **K-Nearest Neighbors (KNN):** As a non-parametric algorithm, KNN classifies new instances based on the majority vote of their  $k$  closest training examples. It relies heavily on the Euclidean distance metric. I tuned the number of neighbors  $k$  and the weight function.
- **Random Forest:** Representing the “Bagging” family, Random Forest constructs a multitude of decision trees during training. By training on bootstrapped subsets of data and using random feature selection at each split, it effectively reduces the variance associated with single decision trees. I tuned the number of estimators and maximum depth.
- **XGBoost (XGB):** Representing the “Boosting” family, Extreme Gradient Boosting builds trees sequentially, where each new tree attempts to correct the errors of the previous ones. XGBoost is widely considered the state-of-the-art for tabular data. I tuned the learning rate, tree depth, and estimator count.

### 2.2 Datasets

I utilized four datasets from the UCI Machine Learning Repository [2], selected to represent a diverse range of classification challenges:

1. **Adult** (Census Income): Predicting if income >\$50k based on heterogeneous census data (age, education, occupation). Contains a mix of continuous and categorical variables.

2. **Letter Recognition:** Identifying the letter “A” from pixel statistics. This is a geometric problem where features represent spatial properties.
3. **Spambase:** Classifying emails as “Spam” or “Ham” based on word frequencies (57 features). Represents high-dimensional tabular data.
4. **Mushroom:** Classifying mushrooms as edible or poisonous. Represents categorical data with deterministic rules.

## 2.3 Experimental Design

My experimental protocol follows a cross-validation framework. For each dataset, I performed the following steps:

1. **Partitioning:** I split the data into three training sizes: 20%, 50%, and 80%, reserving the remainder for testing.
2. **Preprocessing:** Numerical features were standardized (Z-score) and categorical features were One-Hot Encoded.
3. **Tuning:** Within each training partition, I performed 3-fold cross-validation using RandomizedSearchCV to select optimal hyperparameters.
4. **Evaluation:** I reported the final Classification Accuracy on the held-out test set, averaged over 3 independent trials.

## 3. Experimental Results

The following tables summarize the test accuracy for the primary classifiers across the three main datasets.

**Table 1:** Adult Dataset Performance & Accuracy

| Classifier                 | 20% Train | 50% Train | 80% Train |
|----------------------------|-----------|-----------|-----------|
| <i>XGBoost</i>             | 0.855     | 0.862     | 0.867     |
| <i>Random Forest</i>       | 0.851     | 0.860     | 0.858     |
| <i>SVM</i>                 | 0.844     | 0.851     | 0.856     |
| <i>Logistic Regression</i> | 0.845     | 0.849     | 0.852     |
| <i>KNN</i>                 | 0.820     | 0.827     | 0.831     |

**Table 2:** Letter Recognition Performance & Accuracy

| Classifier | 20% Train | 50% Train | 80% Train |
|------------|-----------|-----------|-----------|
|------------|-----------|-----------|-----------|

|                            |       |       |       |
|----------------------------|-------|-------|-------|
| <i>SVM</i>                 | 0.997 | 0.999 | 0.999 |
| <i>KNN</i>                 | 0.996 | 0.998 | 0.999 |
| <i>XGBoost</i>             | 0.996 | 0.998 | 0.998 |
| <i>Random Forest</i>       | 0.994 | 0.996 | 0.997 |
| <i>Logistic Regression</i> | 0.991 | 0.991 | 0.990 |

**Table 3:** Spambase Performance & Accuracy

| <b>Classifier</b>          | <b>20% Train</b> | <b>50% Train</b> | <b>80% Train</b> |
|----------------------------|------------------|------------------|------------------|
| <i>Random Forest</i>       | 0.940            | 0.949            | 0.950            |
| <i>XGBoost</i>             | 0.936            | 0.948            | 0.949            |
| <i>SVM</i>                 | 0.913            | 0.927            | 0.931            |
| <i>Logistic Regression</i> | 0.906            | 0.919            | 0.921            |
| <i>KNN</i>                 | 0.883            | 0.916            | 0.922            |

**Table 4:** Mushroom Dataset Performance & Accuracy

| <b>Classifier</b>          | <b>20% Train</b> | <b>50% Train</b> | <b>80% Train</b> |
|----------------------------|------------------|------------------|------------------|
| <i>Random Forest</i>       | 0.999            | 1.000            | 1.000            |
| <i>XGBoost</i>             | 0.999            | 1.000            | 1.000            |
| <i>SVM</i>                 | 0.999            | 1.000            | 1.000            |
| <i>Logistic Regression</i> | 0.999            | 1.000            | 1.000            |
| <i>KNN</i>                 | 0.998            | 1.000            | 1.000            |

## 4. Discussion & Conclusion

My empirical analysis reconfirms the general consensus in the machine learning community: for structured, tabular data, Gradient Boosting (XGBoost) and Random Forest are the most reliable

classifiers. They offer high accuracy to feature scaling without extensive tuning. However, my results on the Letter Recognition dataset serve as a crucial reminder that methods like SVMs remain highly relevant for domains with geometric or continuous signal properties.

#### **4.1 The Dominance of Ensemble Methods on Tabular Data**

Consistent with the findings in Caruana [1], the ensemble methods, Random Forest and XGBoost, demonstrated superior performance on the Adult and Spambase datasets. These datasets are characterized by tabular heterogeneity, where features have different scales and distributions.

- On **Adult**, XGBoost consistently led the pack with 86.7% accuracy, proving that gradient boosting is particularly effective at handling mixed categorical/numerical data where linear baselines, like Logistic Regression, plateau earlier with 85.2% accuracy.
- On **Spambase**, Random Forest achieved 95.0% accuracy compared to 92.1% for Logistic Regression. This approximate 3% gap is significant in the context of spam filtering.

#### **4.2 The Geometric Advantage of SVMs and KNNs**

While ensembles dominated tabular data, the Letter Recognition dataset provided a distinct counter-example. Here, SVM and KNN achieved an effectively perfect performance of 0.999, slightly outperforming the tree-based ensembles. This dataset represents "geometric" data, pixel statistics that form coherent shapes in vector space. The success of KNN here of 0.999 compared to its struggle on Adult of 0.831 is notable. In the Letter dataset, the Euclidean distance between feature vectors is meaningful, representing visual similarity. In the Adult dataset, the distance between encoded categorical rows is less semantically valid, causing KNN to falter.

#### **4.3 Dataset Saturation**

The Mushroom dataset results indicate that the problem is deterministically solvable. By the 50% training split, every single classifier achieved 100.0% accuracy. Even at 20% training data, the lowest performing model, KNN), achieved 99.8% accuracy. This serves as a control experiment, confirming that our pipeline correctly identifies "easy," separable manifolds where simple rules, captured by Trees, or hyperplanes, captured by SVM/Logistic Regression, work perfectly.

#### **4.4 Impact of Training Set Size**

Across all datasets, I observed a consistent trend: more data leads to better performance, but with diminishing returns.

- **Steep Learning Curve:** The improvement from 20% to 50% training data was often substantial. For example, KNN on Spambase improved from 88.3% to 91.6% (a +3.3% gain).
- **Plateau Effect:** The improvement from 50% to 80% was often marginal. XGBoost on Adult improved only from 86.8% to 86.9%. This suggests that for these standard UCI datasets, 50% of the data is often sufficient for the model to capture the underlying signal. The remaining error is likely irreducible (Bayes error) or due to model bias, rather than variance that could be fixed with more data.

#### 4.5 Bonus Qualification

My study exceeds the standard project requirements to qualify for bonus points in two key areas:

1. **Extended Classifier Scope:** I evaluated 5 distinct classifiers (Random Forest, XGB, SVM, Logistic Regression, KNN), exceeding the minimum of 3. This allowed me to contrast not just different algorithms, but entirely different learning paradigms.
2. **Extended Data Scope:** I utilized 4 datasets (Adult, Letter, Spambase, Mushroom), exceeding the minimum of 3. The inclusion of the Mushroom dataset ensured my pipeline was robust to varying data types.

#### 5. References

[1] Caruana, R., & Niculescu-Mizil, A. (2006). An Empirical Comparison of Supervised Learning Algorithms. Proceedings of the 23rd International Conference on Machine Learning (ICML '06).

[2] UCI Machine Learning Repository. <https://archive.ics.uci.edu/ml>